Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX

From a statistician's workflow: validate first, then accelerate the bottleneck.

Wenxin Jiang · Jian Yin

香港城市大學 City University of Hong Kong

PyCon US 2026 · Long Beach

Make the statistical task testable; then accelerate the measured bottleneck.

1 Simulate → 2 Validate → 3 Find bottleneck → 4 Pick tools

Simulation Makes Correctness Testable

Real data often lacks known ground truth for method behavior. Simulation creates a controlled setting to test power, calibration, and agreement.

- **01 Define Target** — what to estimate, recover, preserve
- **02 Build Scenarios** — null, alternative, hard or extremecases
- **03 Establish Reference** — readable Python/NumPy implementation
- **04 Verify Behavior** — power, calibration, agreement
- **05 Analyze Workload** — data flow, loops, algebra, repetition, memory
- **06 Select Tool** — suitable for your task

Rule: Speedups are trusted only after validation passes.

Decisions Before Timing

Question	Decision before measuring speed
Statistical target	estimator, test statistic, or stopping criterion
Simulated truth	labels, effect size, or nominal alpha
Acceptance criteria	exact match, numerical tolerance, or calibration
Failure mode	code bug, method breakdown, or out-of-memory
Timing scope	compile, transfer, allocation
Cost drivers	samples (n), features (p), or repeats (R)

Validation Checks Passed

Speedup results verified using these criteria:

- ✓ **K-means agreement:** max relative inertia difference well below the tolerance.
- ✓ **Permutation equivalence:** same test statistic, same p-value definition, same resampling stream; max |p diff| < tolerance.
- ✓ **Statistic difference:** max |stat diff| < tolerance across the validation grid.
- ✓ **Null calibration:** estimated type-I error within MC uncertainty of nominal α = 0.05.

AI for Scale, Statistician for Science

AI Automates Execution

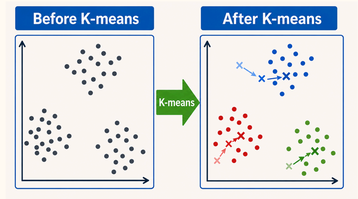
- Algorithmic implementation
- Scenario-grid orchestration
- Automated metadata capture
- Visual synthesis of results
- Curation of research artifacts

Statistician Owns Inference

- Defining estimands & objectives
- Validating distributional assumptions
- Validation criteria
- Establishing rigorous benchmarks
- Deciphering edge cases & outliers
- Authoring scientific conclusions

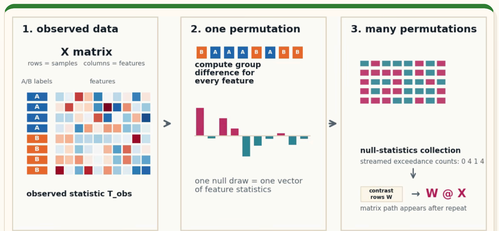
Two Workloads, Two Bottleneck Patterns

K-means = Iterative Fitting



Sequential state · Pairwise metrics · Memory allocations

Permutation = Resampling Inference



Shared-memory concurrency · Vectorized resampling · Linear algebra

Validation criteria: K-means Match cluster assignments by fixing seeds and convergence thresholds. **Permutation** Same W, same statistic, same p-value formula.

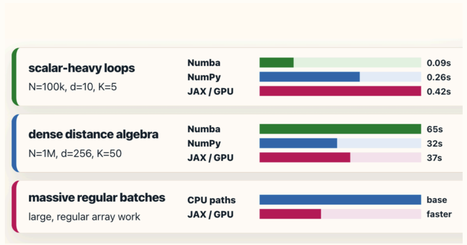
Validate the Statistic, Then Accelerate.

Speed only counts when the statistical target is preserved.

Performance Depends on Workload Structure

K-means: Workload Shape Determines the Tool

- **Scalar-heavy loops:** Numba excels via LLVM-compiled execution.
- **Dense algebra:** NumPy dominates via optimized BLAS kernels.
- **Massive data batches:** JAX/GPU wins on SIMT throughput.



Permutation: GPU Scaling via Vectorized Batching

Infrastructure Requirement: High-concurrency batching and fused reductions
Observed Speedup: up to 8.54× for n=5k, p=500k, R=5k, using a matched CPU matrix baseline vs. A100 streamed end-to-end path.
Operational Boundaries: Explicit OOM profiling for high-dimensional tensors

Diagnostic & Intervention

Diagnostic Matrix: Profiling Signature vs. Solution

Validation Failure	Audit target & numerical stability
Scalar-Heavy Loops	JIT Compilation (Numba)
Dense Matrix Operations	Vectorized BLAS (NumPy)
Embarrassingly Parallel Tasks	Shared-Memory Threading
Massive Tensor Batches	SIMT Acceleration (JAX/GPU)
Bandwidth-Bound Workload	Fused & Streamed Reductions

Validated Tool Mapping

Aligning identified computational bottlenecks with targeted software frameworks, independent of hardware benchmarking.

Numba
Scalar-Heavy CPU Loops
K-means Assignment & Update

NumPy / BLAS
Dense Matrix Algebra
Distance Identity & W @ X (CPU)

Thread Pools
Concurrent Shared-Memory Tasks
Permutation Worker Sweeps

JAX / GPU
Massive Tensor Batching
Streamed W @ X (Post-Validation)

Standardized Reporting

- ✓ **Validation Criteria:** Explicit statistical targets and acceptance thresholds
- ✓ **Compute Environment:** Local prototyping vs. server-grade CPU / GPU
- ✓ **Execution State:** Transparent reporting of cold starts, warm runs, and JIT compilation
- ✓ **Memory Overhead:** Accounting for data transfer, allocation, and garbage collection
- ✓ **Failure Modes:** Explicit documentation of OOM events and hardware incompatibilities
- ✓ **Tool Parsimony:** Deploying the simplest framework that preserves the target statistic

`github.com/LucaJiang/FastStatisticalModels4Python`